



TradeaYa!

SIMULADOR DE TRADING BURSÁTIL

Mercado real • Dinero ficticio • Sistema educativo

Curso

Algorítmica I

Proyecto Final

Docente

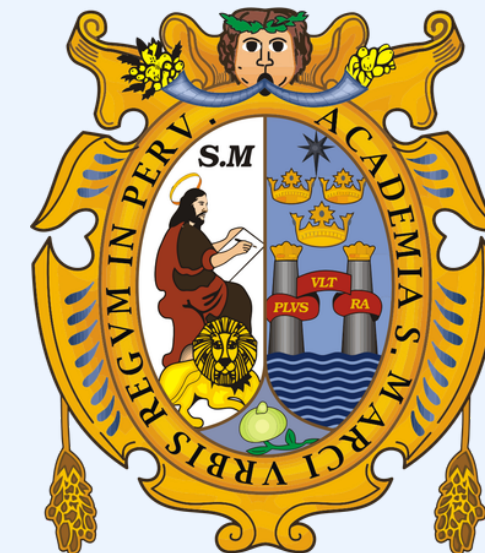
**Arredondo
Gustavo**

Equipo 2

PeruDevs

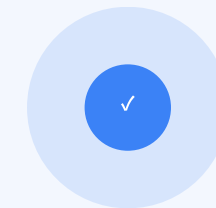
Integrantes

**Abad Cuba David
Sanchez Hinostroza Jose**



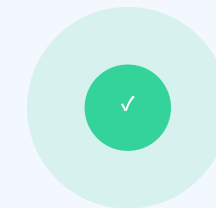
¿Qué es TradeaYa!?

TradeaYa! es una terminal de inversión simulada que conecta al usuario con el mercado real de Estados Unidos - Nueva York y le permite “operar” sobre activos financieros con dinero ficticio mientras analiza gráficos.



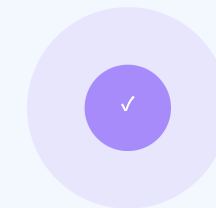
Dinero ficticio - Mercado real

El sistema se enfoca en el mercado de valores de Nueva York. El dinero utilizado no es real.



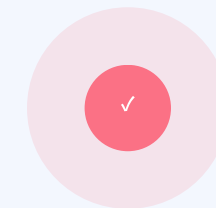
Modularidad

Cada parte del sistema está en su debida carpeta. Las funciones se llaman de otros archivos para formar parte de nuevas funciones.



Datos en vivo

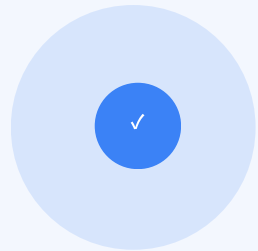
Actualización automática cada 60 segundos para garantizar precios actuales.



Desplegado en la nube

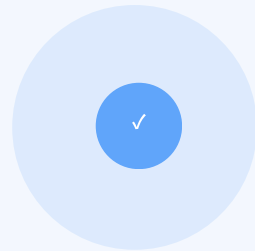
Streamlit Cloud + Base de datos Supabase para respaldar la interacción de usuarios independientemente de su sistema.

Stack tecnológico



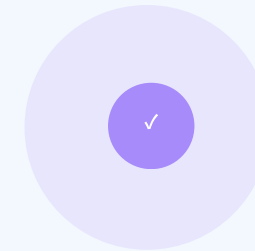
Python

Lenguaje base de todo el sistema;
elegido por su versatilidad y amplia
gama de librerías



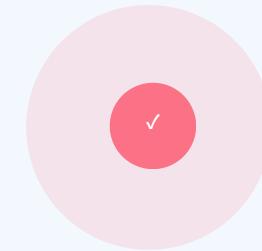
Streamlit

Framework web reactivo,
importado como librería, que crea la
interfaz del programa



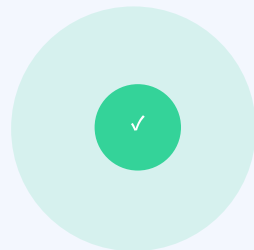
SQLite / PostgreSQL

Importación de la librería SQLite3 y
conexión con una base de datos
cloud en Supabase



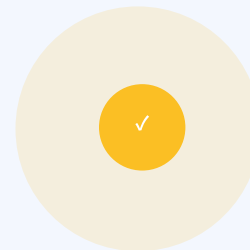
Plotly

Librería que permite realizar
gráficos de velas japonesas e
indicadores necesarios en
aplicaciones de Trading



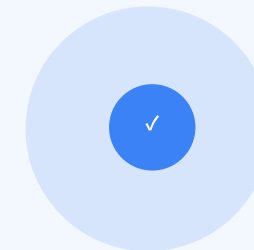
Yahoo Finance

Importado como librería para
obtener precios actuales y datos
históricos de mercado



Streamlit Cloud

Plataforma gratuita que permite
desplegar aplicaciones web creadas
con Streamlit



Pandas / Numpy

Librerías estándar para manejar
tablas y procesar series temporales
mediante DataFrame

Arquitectura basada en tres capas

Cada capa está separada por carpeta; todas proporcionan algo vital para el sistema.



FRONTEND

`src/frontend/`

Renderizado de toda la interfaz con Streamlit.

Botones, refresco de datos, interacciones: todo lo necesario para apelar al usuario.



LOGIC

`src/logic/`

Reglas de negocio puras. Aquí yacen los cimientos de las reglas de negocio y los parámetros por los que TODO el sistema se rige.



BACKEND

`src/backend/`

Creación de conexiones para obtener y administrar datos de Yahoo Finance, así como la definición de ambas bases de datos (SQLite/PostgreSQL).

Frontend --> Logic --> Backend (Nota: Todo dato de mercado pasa por TradeEngine)

TradeEngine: puente único con el mercado

Toda consulta de precios o históricos pasa por esta clase. Ninguna otra parte del código llama directamente a yfinance.

```
class TradeEngine:

    def obtenerPrecioActual(self, ticker):
        return yf.Ticker(ticker)
            .fast_info['last_price']
        # Se llama cada 60s desde la UI

    def obtener_datos_grafico(self, ticker,
                             periodo='1mo'):
        return hist
        # Historial OHLCV para velas y ML

    def validar_ticker(self, ticker):
        return bool(yf.Ticker(ticker).info)
        # True si existe el ticker, False si no
```

obtenerPrecioActual

Obtención del precio en vivo más reciente registrado en Yahoo Finance

obtener_datos_grafico

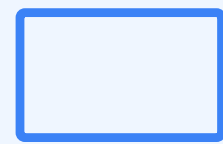
Recolección de datos históricos de una acción OHLCV (1 día, 1 mes, 1 año)

validar_ticker

Verifica si un símbolo existe

Persistencia dual automática

El sistema detecta el entorno de ejecución y adapta la conexión sin cambiar la lógica del código.



Modo local: SQLite

Mediante la importación de la librería SQLite3, se crea automáticamente el archivo tradeaya.db

Se activa sólo cuando no existe la variable DATABASE_URL o falla la conexión con la nube. De todas maneras, su información persiste y se usa.



Modo nube: PostgreSQL

En Supabase, se crea una Data Base de plan gratuito y se enlaza via DATABASE_URL en Streamlit Secrets

Implementación: respuesta a la necesidad de persistencia real entre sesiones y despliegues → El archivo tradeaya.db puede brindar información, pero no se actualiza cuando se utiliza el sistema desde la web; Supabase sí lo hace.

Tres tablas relacionales: usuarios, portafolio y transacciones

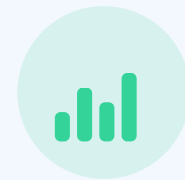
Reglas de negocio puras

Funciones puras. Utilización de pytz, time y datetime.



Horario de mercado

Lunes a viernes, 9:30 a 16:00 hora de Nueva York.



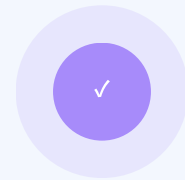
Comisión de broker

0.5% aplicada sobre cada operación de compra/venta.



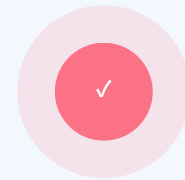
Vigencia del precio

Máximo 60 segundos de antigüedad en los precios.



Validación de compra

¿El saldo es suficiente? ¿El mercado está abierto? ¿El precio está vigente? Si la respuesta es sí, entonces adelante.



Validación de venta

¿Tienes este ticker? ¿Lo posees en cantidad suficiente?
Si es así, puedes vender.



Cálculo de rendimiento

$(\text{Precio actual} - \text{Precio de compra}) / \text{Precio de compra}$

SesionTrading: la sesión del usuario

Une portafolio y cálculos en un único objeto con estado; el Frontend solo necesita conocer esta clase, ya que guarda el saldo, portafolio e historial.

```
class SesionTrading:
    saldo = 10_000.00
    portafolio: dict
    historial: list[dict]

    def comprar(...): ...
    def vender(...): ...
```

registrar_compra()

Recibe la orden, la valida y actualiza calculando el precio de compra promedio ponderado por cantidad en cada nueva compra del mismo ticker.

registrar_venta()

Recibe la orden y la valida. Actualiza de la siguiente manera: venta total elimina la posición, venta parcial reduce la cantidad y conserva el precio promedio.

Flujo general de la aplicación

1

Login / registro

Registro/inicio de sesión con un correo y contraseña. NO incluye verificación de e-mail ni funcionalidades de cuenta.

2

Restauración de sesión

El sistema persiste en la nube: aún si se reinicia la página, el identificador de usuario sobrevive y este puede seguir interactuando.

3

Terminal (dashboard)

Dos pestañas con las que el usuario interactúa: Portafolio y Zona de Inversión.

<https://tradeaya.streamlit.app/>

Terminal de inversión: 2 pestañas



Portafolio

Revisa las métricas de tu portafolio

```
def _mostrar_portafolio():  
    # Muestra la tabla de  
    posiciones abiertas  
  
def grafico_portafolio_live():  
    # Grafico de una posicion del  
    portafolio  
  
def _mostrar_historial():  
    # Muestra el historial de  
    transacciones
```



Zona de inversión

Revisa y opera con acciones

```
def grafico_inversion_live():  
    # Grafico del ticker con  
    botones y calculadora
```

_mostrar_grafico()

Grafico histórico de una acción

Datos en vivo sin recargar la página

Streamlit permite refrescar solo una porción de la interfaz, de forma autónoma y periódica.

60s

grafico_portafolio_live() Métricas, tabla y gráfico del portafolio

60s

grafico_inversion_live() Gráfico, precio, calculadora y botones de operación

60s

saldo_sidebar_live() Saldo disponible en la barra lateral

1s

reloj_mercado_en_vivo() Estado del mercado (abierto/cerrado) y hora de Nueva York

¡Muchas Gracias!

PeruDevs • Equipo 2

